Student Task Manager

Student name: DIXITA NEMA
Registration Number: 24BAI10802
Course: Programming in Java
Institution: VITyarthi
Submission Date: 31-03-26

Slot: B21+E14

Introduction

Student life is full of assignments, homework, project deadlines, class tests, practical submissions, and many other academic activities. Many students struggle to manage all these activities properly because they often forget important tasks or fail to remember deadlines.

Nowadays, many people use mobile apps or websites for task management, but not every student wants a complicated application. Some students need a simple and lightweight system that can work directly on a computer without internet access.

The Student Task Manager project is a simple Java-based application that helps students organize their daily academic work. It allows users to add tasks, view tasks, mark tasks as completed, and delete unnecessary tasks. The project also stores tasks permanently in a text file so that the data is not lost when the program is closed.

This project is built using Java programming concepts such as classes, objects, methods, ArrayList, loops, conditionals, and file handling.

Problem Statement

Students often forget assignments, homework, projects, and deadlines because they do not maintain proper records of their work.

Some common problems faced by students are:

- Missing assignment deadlines
- Forgetting project submissions
- Poor time management
- Difficulty in tracking completed and pending work
- No proper way to store tasks permanently

This project aims to solve these problems by providing a simple task management system.

Need for the Project

The need for this project comes from the daily challenges students face while managing studies.

Students usually note tasks in notebooks or on paper, but those methods are not very reliable because:

- Papers can get lost
- Notes can be forgotten
- Tasks are not easy to update
- There is no way to track completion status

A digital task manager solves these issues because it allows students to:

- Store tasks safely
- View all tasks in one place
- Mark completed work
- Remove unnecessary tasks
- Save tasks permanently

This makes academic planning easier and improves productivity.

Scope of the Project

The scope of this project is limited to task management for students.

The project can be used by:

- School students
- College students
- Coaching students
- Anyone who wants to manage simple tasks

Currently, the project is a console-based application, but in the future it can be upgraded into:

- A GUI application
- A mobile application
- A web application
- A cloud-based task manager

Technologies Used

The following technologies were used in this project:

- **Java** Java is the main programming language used to build this project.
- **VS Code** VS Code was used as the code editor.
- **GitHub** GitHub was used to upload the project repository and maintain version control.
- **Text File** A text file named tasks.txt was used to save task data.

Concepts Used in Java

This project uses many Java concepts taught in the course.

Classes and Objects - Classes are used to create the structure of the project. Objects are used to store task details. Example:

- Task class
- TaskManager class
- Main class

Encapsulation - Data such as task title, deadline, and priority are stored inside objects.

ArrayList - ArrayList is used to store multiple tasks dynamically.

Loops - Loops are used to display tasks repeatedly.

Conditional Statements - If-else conditions are used to validate task numbers.

Switch Case - Switch case is used for the menu-driven system.

File Handling - BufferedReader and BufferedWriter are used to save and load tasks.

System Requirements

The following software requirements are needed:

Software Requirements

- Java JDK 17 or above
- VS Code
- Git
- Internet connection for GitHub upload

Project Structure

The project contains the following files:

StudentTaskManager/

- Main.java
- Task.java
- TaskManager.java
- README.md
- tasks.txt

Main.java - Contains the main menu and user interaction.

Task.java - Stores information about tasks.

TaskManager.java - Contains the methods for adding, viewing, deleting, saving, and loading tasks.

tasks.txt - Stores task details permanently.

README.md - Contains the explanation of the project for GitHub.

Detailed Explanation of Classes

Task Class

The Task class is responsible for storing information related to each task. It contains the following variables:

- title
- deadline
- priority
- isCompleted

It also contains methods such as:

- Constructor
- markCompleted()
- toString()
- fromString()

The constructor initializes task details when a new task is created. The markCompleted() method changes the completion status from false to true. The toString() method converts the task into a text format for storing in a file. The fromString() method converts file data back into a task object.

Code:

```
class Task {
    String title;
    String deadline;
    String priority;
    boolean isCompleted;

    public Task(String title, String deadline, String priority) {
        this.title = title;
        this.deadline = deadline;
        this.priority = priority;
        this.isCompleted = false;
    }

    public void markCompleted() {
        isCompleted = true;
    }

    @Override
    public String toString() {
        return title + "," + deadline + "," + priority + "," + isCompleted;
    }
```

```
    public static Task fromString(String data) {
        String[] parts = data.split(",");
        Task task = new Task(parts[0], parts[1], parts[2]);
        task.isCompleted = Boolean.parseBoolean(parts[3]);
        return task;
    }
}
```

TaskManager Class

The TaskManager class manages all operations related to tasks. It contains an ArrayList of Task objects. Important methods:

- addTask()
- viewTasks()
- markTaskCompleted()
- deleteTask()
- saveToFile()
- loadFromFile()

Code:

```
import java.io.*;
import java.util.ArrayList;

class TaskManager {
    ArrayList<Task> tasks = new ArrayList<>();

    public void addTask(String title, String deadline, String priority) {
        tasks.add(new Task(title, deadline, priority));
    }

    public void viewTasks() {
        if (tasks.isEmpty()) {
            System.out.println("No tasks available.");
            return;
```

```
    }

    for (int i = 0; i < tasks.size(); i++) {
        Task t = tasks.get(i);
        System.out.println((i + 1) + ". " + t.title +
                " | Deadline: " + t.deadline +
                " | Priority: " + t.priority +
                " | Completed: " + t.isCompleted);
    }
}

public void markTaskCompleted(int index) {
    if (index >= 0 && index < tasks.size()) {
        tasks.get(index).markCompleted();
        System.out.println("Task marked as completed.");
    } else {
        System.out.println("Invalid task number.");
    }
}

public void deleteTask(int index) {
    if (index >= 0 && index < tasks.size()) {
        tasks.remove(index);
        System.out.println("Task deleted successfully.");
    } else {
        System.out.println("Invalid task number.");
    }
}

public void loadFromFile() throws Exception {
    File file = new File("tasks.txt");
```

```
        if (!file.exists()) {
            return;
        }

        try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = reader.readLine()) != null) {
                tasks.add(Task.fromString(line));
            }
        }
    }

    public void saveToFile() throws Exception {
        try (BufferedWriter bw = new BufferedWriter(new FileWriter("tasks.txt"))) {
            for (Task t : tasks) {
                bw.write(t.toString());
                bw.newLine();
            }
        }
    }
}
```

Main Class

The Main class contains the main method. It displays the menu options and takes user input. The switch case handles all the operations.

Code:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);
TaskManager manager = new TaskManager();
try {
    manager.loadFromFile();
} catch (Exception e) {
    System.out.println("Error loading tasks.");
}

while (true) {
    System.out.println("\n--- Student Task Manager ---");
    System.out.println("1. Add Task");
    System.out.println("2. View Tasks");
    System.out.println("3. Mark Task Completed");
    System.out.println("4. Delete Task");
    System.out.println("5. Exit");

    int choice = sc.nextInt();
    sc.nextLine();

    switch (choice) {
        case 1 -> {
            System.out.print("Enter title: ");
            String title = sc.nextLine();

            System.out.print("Enter deadline: ");
            String deadline = sc.nextLine();

            System.out.print("Enter priority: ");
            String priority = sc.nextLine();

            manager.addTask(title, deadline, priority);
```

```
            }
            case 2 -> manager.viewTasks();
            case 3 -> {
                System.out.print("Enter task number: ");
                int index = sc.nextInt() - 1;
                manager.markTaskCompleted(index);
            }
            case 4 -> {
                System.out.print("Enter task number: ");
                int del = sc.nextInt() - 1;
                manager.deleteTask(del);
            }
            case 5 -> {
                try {
                    manager.saveToFile();
                } catch (Exception e) {
                    System.out.println("Error saving tasks.");
                }
                System.out.println("Tasks saved successfully.");
                System.out.println("Exiting program...");
                sc.close();
                return;
            }
            default -> System.out.println("Invalid choice!");

        }
    }
  }
}
```

Working of the Project

The working of the project is simple.

1. The program starts.
2. Existing tasks are loaded from tasks.txt.
3. A menu is displayed.
4. User selects an option.
5. Depending on the choice:
    - Task is added
    - Task is viewed
    - Task is marked completed
    - Task is deleted
6. When the user exits, tasks are saved in tasks.txt.

Features of the Project

The Student Task Manager project contains the following features:

- **Add Task -** The user can add a new task with title, deadline, and priority.
- **View Tasks -** The user can view all tasks in a numbered list.
- **Mark Task as Completed -** The user can mark a task as completed.
- **Delete Task -** The user can remove tasks that are no longer needed.
- **Save Tasks -** Tasks are saved in tasks.txt.
- **Load Tasks Automatically -** Tasks are automatically loaded when the program starts.

File Handling Process

File handling is one of the most important parts of this project. Without file handling, tasks would disappear when the program closes. The saveToFile() method writes all task details into tasks.txt.

Example: Java Assignment,5 April,High,false DBMS Homework,10 April,Medium,true. The loadFromFile() method reads this data and recreates task objects. This ensures that data is not lost.

Sample Outputs

Main Menu

- Add Task
- View Tasks
- Mark Task as Completed
- Delete Task
- Exit

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Dixita Nema\OneDrive\Desktop\Student TaskManager> javac *.java
>> java Main

--- Student Task Manager ---
1. Add Task
2. View Tasks
3. Mark Task Completed
4. Delete Task
5. Exit
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Dixita Nema\OneDrive\Desktop\Student TaskManager> javac *.java
>> java Main
1
Enter title: Java Practical
Enter deadline: 4th April
Enter priority: 1

--- Student Task Manager ---
1. Add Task
2. View Tasks
3. Mark Task Completed
4. Delete Task
5. Exit
2
1. Java Practical | Deadline: 4th April | Priority: 1 | Completed: false
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Dixita Nema\OneDrive\Desktop\Student TaskManager> javac *.java
>> java Main
5. Exit
3
Enter task number: 1
Task marked as completed.

--- Student Task Manager ---
1. Add Task
2. View Tasks
3. Mark Task Completed
4. Delete Task
5. Exit
2
1. Java Practical | Deadline: 4th April | Priority: 1 | Completed: true
```

```
--- Student Task Manager ---
1. Add Task
2. View Tasks
3. Mark Task Completed
4. Delete Task
5. Exit
5
Tasks saved successfully.
Exiting program...
PS C:\Users\Dixita Nema\OneDrive\Desktop\Student TaskManager>
```

Challenges Faced

During the development of this project, several challenges were faced.

- Understanding Java file handling
- Managing ArrayList correctly
- Fixing case-sensitive file names
- Connecting GitHub repository
- Solving push and pull errors in Git
- Creating a proper README file

These challenges helped improve problem-solving skills.

Testing and Results

The project was tested multiple times. Different tasks were added, viewed, marked completed, and deleted. The file saving feature was also tested by closing and reopening the application.

The results were successful because:

- Tasks were added correctly
- Tasks were displayed properly
- Completed tasks showed correct status
- Deleted tasks were removed successfully
- Data was stored permanently in tasks.txt

Future Enhancements

This project can be improved in many ways in the future.

Possible future improvements are:

- GUI version using Java Swing
- User login system
- Task notifications
- Sorting by priority
- Search feature
- Due date reminders
- Mobile app version
- Database integration

Conclusion

The Student Task Manager project is a useful and practical Java application. It helps students organize assignments and deadlines in a better way. The project uses important Java concepts such as classes, objects, methods, ArrayList, loops, conditionals, and file handling. The project is simple, easy to understand, and effective for daily use.

It also provides a strong foundation for future improvements such as GUI development and database integration. Overall, this project successfully achieves its objectives and demonstrates the practical application of Java programming.

References:

- Java Documentation
- VS Code Docume